

CEI Summer Internship 2026

Week 2 Task: E-Commerce Sales Database

```
-- PostgreSQL

--Customers Table
CREATE TABLE customers (
  customer_id INT PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  city VARCHAR(50) NOT NULL,
  state VARCHAR(50) NOT NULL,
  join_date DATE NOT NULL,
  is_premium BOOLEAN DEFAULT FALSE
);
CREATE INDEX idx_customers_city ON customers(city);
CREATE INDEX idx_customers_state ON customers(state);

--Product Table
CREATE TABLE products (
  product_id INT PRIMARY KEY,
  product_name VARCHAR(100) NOT NULL,
  category VARCHAR(50) NOT NULL,
  brand VARCHAR(50) NOT NULL,
  unit_price DECIMAL(10,2) NOT NULL CHECK (unit_price > 0),
  stock_qty INT NOT NULL DEFAULT 0 CHECK (stock_qty >= 0)
);
CREATE INDEX idx_products_category ON products(category);

--Orders Table
CREATE TABLE orders (
  order_id INT PRIMARY KEY,
  customer_id INT NOT NULL,
  order_date DATE NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'Pending'
  CHECK (status IN ('Pending','Shipped','Delivered','Cancelled')),
  total_amount DECIMAL(12,2) NOT NULL CHECK (total_amount >= 0),
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
CREATE INDEX idx_orders_date ON orders(order_date);
CREATE INDEX idx_orders_status ON orders(status);

--Order_items Table
CREATE TABLE order_items (
  item_id INT PRIMARY KEY,
  order_id INT NOT NULL,
  product_id INT NOT NULL,
  quantity INT NOT NULL CHECK (quantity > 0),
  unit_price DECIMAL(10,2) NOT NULL CHECK (unit_price > 0),
  discount_pct DECIMAL(5,2) DEFAULT 0 CHECK (discount_pct BETWEEN 0 AND
100),
  FOREIGN KEY (order_id) REFERENCES orders(order_id),
  FOREIGN KEY (product_id) REFERENCES products(product_id)
```

```
);
INSERT INTO customers VALUES
(101, 'Aarav', 'Sharma', 'aarav.s@email.com', 'Mumbai',
'Maharashtra', '2024-01-15', TRUE),
(102, 'Priya', 'Patel', 'priya.p@email.com', 'Ahmedabad', 'Gujarat',
'2024-02-20', FALSE),
(103, 'Rohan', 'Gupta', 'rohan.g@email.com', 'Delhi', 'Delhi',
'2024-03-10', TRUE),
(104, 'Sneha', 'Reddy', 'sneha.r@email.com', 'Hyderabad', 'Telangana',
'2024-04-05', FALSE),
(105, 'Vikram', 'Singh', 'vikram.s@email.com', 'Jaipur', 'Rajasthan',
'2024-05-12', TRUE),
(106, 'Ananya', 'Iyer', 'ananya.i@email.com', 'Chennai', 'Tamil
Nadu', '2024-06-18', FALSE),
(107, 'Karan', 'Mehta', 'karan.m@email.com', 'Pune',
'Maharashtra', '2024-07-22', TRUE),
(108, 'Divya', 'Nair', 'divya.n@email.com', 'Kochi', 'Kerala',
'2024-08-30', FALSE);
```

```
INSERT INTO products VALUES
(201, 'Wireless Earbuds', 'Electronics', 'BoAt', 1499.00,
250),
(202, 'Cotton T-Shirt', 'Clothing', 'Levis', 799.00,
500),
(203, 'Smart Watch', 'Electronics', 'Noise', 2999.00,
150),
(204, 'Running Shoes', 'Clothing', 'Nike', 4599.00,
120),
(205, 'Bluetooth Speaker', 'Electronics', 'JBL', 3499.00,
200),
(206, 'Bedsheet Set', 'Home', 'Spaces', 1299.00,
300),
(207, 'Laptop Stand', 'Electronics', 'AmazonBasics', 899.00,
180),
(208, 'Cushion Covers (Set)', 'Home', 'HomeCenter', 599.00,
400);
```

```
INSERT INTO orders VALUES
(1001, 101, '2024-08-01', 'Delivered', 4498.00),
(1002, 102, '2024-08-03', 'Delivered', 799.00),
(1003, 103, '2024-08-05', 'Shipped', 7498.00),
(1004, 101, '2024-08-10', 'Delivered', 3499.00),
(1005, 104, '2024-08-12', 'Cancelled', 2999.00),
(1006, 105, '2024-08-15', 'Delivered', 5898.00),
(1007, 106, '2024-08-18', 'Pending', 1299.00),
(1008, 103, '2024-08-20', 'Delivered', 899.00),
(1009, 107, '2024-08-25', 'Shipped', 6098.00),
(1010, 108, '2024-08-28', 'Delivered', 1598.00);
```

```
INSERT INTO order_items VALUES
(5001, 1001, 201, 2, 1499.00, 0),
(5002, 1001, 207, 1, 899.00, 10),
(5003, 1002, 202, 1, 799.00, 0),
(5004, 1003, 203, 1, 2999.00, 0),
(5005, 1003, 204, 1, 4599.00, 5),
(5006, 1004, 205, 1, 3499.00, 0),
```

```
(5007, 1005, 203, 1, 2999.00, 0),
(5008, 1006, 201, 1, 1499.00, 10),
(5009, 1006, 204, 1, 4599.00, 5),
(5010, 1007, 206, 1, 1299.00, 0),
(5011, 1008, 207, 1, 899.00, 0),
(5012, 1009, 205, 1, 3499.00, 0),
(5013, 1009, 208, 2, 599.00, 15),
(5014, 1010, 206, 1, 1299.00, 0),
(5015, 1010, 208, 1, 599.00, 0);
```

-- Q1

```
SELECT * FROM customers;
```

Output:

	customer_id [PK] integer	first_name character varying (50)	last_name character varying (50)	email character varying (100)	city character varying (50)	state character varying (50)	join_date date	is_premium boolean
1	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	true
2	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	false
3	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	true
4	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	false
5	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	true
6	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	false
7	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	true
8	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	false

-- Q2

```
SELECT first_name, last_name, city
FROM customers;
```

Output:

	first_name character varying (50)	last_name character varying (50)	city character varying (50)
1	Aarav	Sharma	Mumbai
2	Priya	Patel	Ahmedabad
3	Rohan	Gupta	Delhi
4	Sneha	Reddy	Hyderabad
5	Vikram	Singh	Jaipur
6	Ananya	Iyer	Chennai
7	Karan	Mehta	Pune
8	Divya	Nair	Kochi

-- Q3

```
SELECT DISTINCT category
FROM products;
```

Output:

	category character varying (50) 
1	Electronics
2	Home
3	Clothing

```
-- Q4
-- Primary Keys:
-- customers -> customer_id
-- products  -> product_id
-- orders    -> order_id
-- order_items -> item_id
--
-- A Primary Key uniquely identifies each record.
-- It must be UNIQUE and NOT NULL to prevent duplicate
-- and missing identifiers.

-- Q5
-- Constraints on email:
-- UNIQUE NOT NULL
--
-- Attempting to insert a duplicate email will result in:
-- ERROR: duplicate key value violates unique constraint
```

```
-- Example:
INSERT INTO customers
VALUES (
109,
'Test',
'User',
'aarav.s@email.com',
'Mumbai',
'Maharashtra',
'2024-09-01',
FALSE
);
```

Output:

```
ERROR:  duplicate key value violates unique constraint "customers_email_key"
Key (email)=(aarav.s@email.com) already exists.

SQL state: 23505
Detail: Key (email)=(aarav.s@email.com) already exists.
```

```
-- Q6
-- CHECK Constraint Example
```

```
INSERT INTO products
VALUES (
209,
'Bad Product',
```

```
'Electronics',
'XYZ',
-50,
10
);

-- Expected Error:
-- ERROR: new row violates check constraint
-- Constraint:
-- CHECK (unit_price > 0)
```

```
Output:
ERROR:  new row for relation "products" violates check constraint "products_unit_price_check"
Failing row contains (209, Bad Product, Electronics, XYZ, -50.00, 10).

SQL state: 23514
Detail: Failing row contains (209, Bad Product, Electronics, XYZ, -50.00, 10).
```

```
-- SECTION B

-- Q7
SELECT *
FROM orders
WHERE status = 'Delivered';
```

	order_id [PK] integer	customer_id integer	order_date date	status character varying (20)	total_amount numeric (12,2)
1	1001	101	2024-08-01	Delivered	4498.00
2	1002	102	2024-08-03	Delivered	799.00
3	1004	101	2024-08-10	Delivered	3499.00
4	1006	105	2024-08-15	Delivered	5898.00
5	1008	103	2024-08-20	Delivered	899.00
6	1010	108	2024-08-28	Delivered	1598.00

```
-- Q8
SELECT *
FROM products
WHERE category = 'Electronics'
AND unit_price > 2000;
```

	product_id [PK] integer	product_name character varying (100)	category character varying (50)	brand character varying (50)	unit_price numeric (10,2)	stock_qty integer
1	203	Smart Watch	Electronics	Noise	2999.00	150
2	205	Bluetooth Speaker	Electronics	JBL	3499.00	200

```
-- Q9
SELECT *
FROM customers
WHERE join_date BETWEEN '2024-01-01' AND '2024-12-31'
AND state = 'Maharashtra';
```

	customer_id [PK] integer	first_name character varying (50)	last_name character varying (50)	email character varying (100)	city character varying (50)	state character varying (50)	join_date date	is_premium boolean
1	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	true
2	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	true

```
-- Q10
SELECT *
FROM orders
WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'
AND status <> 'Cancelled';
```

	order_id [PK] integer	customer_id integer	order_date date	status character varying (20)	total_amount numeric (12,2)
1	1004	101	2024-08-10	Delivered	3499.00
2	1006	105	2024-08-15	Delivered	5898.00
3	1007	106	2024-08-18	Pending	1299.00
4	1008	103	2024-08-20	Delivered	899.00
5	1009	107	2024-08-25	Shipped	6098.00

```
-- Q11
-- Index: idx_orders_date
-- It improves performance of searches, filtering,
-- and sorting based on order_date.
-- Example :
```

```
SELECT *
FROM orders
WHERE order_date BETWEEN '2024-08-09'
AND '2024-08-21';
```

	order_id [PK] integer	customer_id integer	order_date date	status character varying (20)	total_amount numeric (12,2)
1	1004	101	2024-08-10	Delivered	3499.00
2	1005	104	2024-08-12	Cancelled	2999.00
3	1006	105	2024-08-15	Delivered	5898.00
4	1007	106	2024-08-18	Pending	1299.00
5	1008	103	2024-08-20	Delivered	899.00

```
-- Q12
-- Query using function on column:
```

```
SELECT *
FROM customers
WHERE EXTRACT(YEAR FROM join_date) = 2023;
```

```
-- The index would generally not be used efficiently
-- because a function is applied to the indexed column.
--
-- SARGable version:
```

```
SELECT *
FROM customers
WHERE join_date >= '2023-11-01'
AND join_date < '2025-01-01';
```

	customer_id	first_name	last_name	email	city	state	join_date	is_premium
	[PK] integer	character varying (50)	character varying (50)	character varying (100)	character varying (50)	character varying (50)	date	boolean
1	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	true
2	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	false
3	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	true
4	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	false
5	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	true
6	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	false
7	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	true
8	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	false

-- SECTION C

```
-- Q13
SELECT COUNT(*) AS total_orders
FROM orders;
```

	total_orders	
	bigint	
1		10

```
-- Q14
SELECT SUM(total_amount) AS total_revenue
FROM orders
WHERE status = 'Delivered';
```

	total_revenue	
	numeric	
1		17191.00

```
-- Q15
SELECT
category,
AVG(unit_price) AS avg_unit_price
FROM products
GROUP BY category;
```

	category character varying (50) 🔒	avg_unit_price numeric 🔒
1	Electronics	2224.000000000000000000
2	Home	949.000000000000000000
3	Clothing	2699.000000000000000000

```
-- Q16
SELECT
status,
COUNT(*) AS order_count,
SUM(total_amount) AS total_revenue
FROM orders
GROUP BY status
ORDER BY total_revenue DESC;
```

	status character varying (20) 🔒	order_count bigint 🔒	total_revenue numeric 🔒
1	Delivered	6	17191.00
2	Shipped	2	13596.00
3	Cancelled	1	2999.00
4	Pending	1	1299.00

```
-- Q17
SELECT
category,
MAX(unit_price) AS highest_price,
MIN(unit_price) AS lowest_price
FROM products
GROUP BY category;
Output:
```

	category character varying (50) 🔒	highest_price numeric 🔒	lowest_price numeric 🔒
1	Electronics	3499.00	899.00
2	Home	1299.00	599.00
3	Clothing	4599.00	799.00

```
-- Q18
```



```

SELECT
category,
AVG(unit_price) AS avg_price
FROM products
GROUP BY category
HAVING AVG(unit_price) > 2000;
Output:

```

	category character varying (50) 🔒	avg_price numeric 🔒
1	Electronics	2224.0000000000000000
2	Clothing	2699.0000000000000000

```
-- SECTION D
```

```

-- Q19
SELECT
o.order_id,
o.order_date,
c.first_name,
c.last_name,
o.total_amount
FROM orders o
INNER JOIN customers c
ON o.customer_id = c.customer_id;
Output:

```

	order_id integer 🔒	order_date date 🔒	first_name character varying (50)	last_name character varying (50) 🔒	total_amount numeric (12,2) 🔒
1	1001	2024-08-01	Aarav	Sharma	4498.00
2	1002	2024-08-03	Priya	Patel	799.00
3	1003	2024-08-05	Rohan	Gupta	7498.00
4	1004	2024-08-10	Aarav	Sharma	3499.00
5	1005	2024-08-12	Sneha	Reddy	2999.00
6	1006	2024-08-15	Vikram	Singh	5898.00
7	1007	2024-08-18	Ananya	Iyer	1299.00
8	1008	2024-08-20	Rohan	Gupta	899.00
9	1009	2024-08-25	Karan	Mehta	6098.00
10	1010	2024-08-28	Divya	Nair	1598.00

```

-- Q20
SELECT
c.customer_id,
c.first_name,
c.last_name,
o.order_id,

```

```

o.order_date
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
Output:

```

	customer_id integer	first_name character varying (50)	last_name character varying (50) 🔒	order_id integer 🔒	order_date date 🔒
1	101	Aarav	Sharma	1001	2024-08-01
2	102	Priya	Patel	1002	2024-08-03
3	103	Rohan	Gupta	1003	2024-08-05
4	101	Aarav	Sharma	1004	2024-08-10
5	104	Sneha	Reddy	1005	2024-08-12
6	105	Vikram	Singh	1006	2024-08-15
7	106	Ananya	Iyer	1007	2024-08-18
8	103	Rohan	Gupta	1008	2024-08-20
9	107	Karan	Mehta	1009	2024-08-25
10	108	Divya	Nair	1010	2024-08-28

```

-- Q21
SELECT
o.order_id,
p.product_name,
oi.quantity,
oi.unit_price,
oi.discount_pct
FROM orders o
JOIN order_items oi
ON o.order_id = oi.order_id
JOIN products p
ON oi.product_id = p.product_id;

```

Output:

	order_id integer 🔒	product_name character varying (100) 🔒	quantity integer 🔒	unit_price numeric (10,2) 🔒	discount_pct numeric (5,2) 🔒
1	1001	Wireless Earbuds	2	1499.00	0.00
2	1001	Laptop Stand	1	899.00	10.00
3	1002	Cotton T-Shirt	1	799.00	0.00
4	1003	Smart Watch	1	2999.00	0.00
5	1003	Running Shoes	1	4599.00	5.00
6	1004	Bluetooth Speaker	1	3499.00	0.00
7	1005	Smart Watch	1	2999.00	0.00
8	1006	Wireless Earbuds	1	1499.00	10.00
9	1006	Running Shoes	1	4599.00	5.00
10	1007	Bedsheet Set	1	1299.00	0.00
11	1008	Laptop Stand	1	899.00	0.00
12	1009	Bluetooth Speaker	1	3499.00	0.00
13	1009	Cushion Covers (Set)	2	599.00	15.00
14	1010	Bedsheet Set	1	1299.00	0.00
15	1010	Cushion Covers (Set)	1	599.00	0.00

```
-- Q22
-- LEFT JOIN: Returns all rows from left table and matching rows
-- from right table.
-- RIGHT JOIN: Returns all rows from right table and matching rows
-- from left table.
--
-- Example:customers LEFT JOIN orders
-- returns all customers even if no orders exist.
-- customers RIGHT JOIN orders
-- returns all orders even if customer record is missing.
--
-- FULL OUTER JOIN:Returns matching rows plus unmatched rows
-- from both tables.

-- Q23
-- Foreign Keys:
-- orders.customer_id
-- REFERENCES customers(customer_id)
-- order_items.order_id
-- REFERENCES orders(order_id)
-- order_items.product_id
-- REFERENCES products(product_id)
-- Example:

INSERT INTO orders
VALUES (
1011,
```

```

109,
CURRENT_DATE,
'Pending',
1000
);
-- Expected:
-- ERROR: insert or update on table "orders"
-- violates foreign key constraint

```

Output:

```

ERROR: insert or update on table "orders" violates foreign key constraint "orders_customer_id_fkey"
Key (customer_id)=(109) is not present in table "customers".

SQL state: 23503
Detail: Key (customer_id)=(109) is not present in table "customers".

```

```
-- SECTION E
```

```

-- Q24
SELECT
product_name,
unit_price,
CASE
    WHEN unit_price < 1000 THEN 'Budget'
    WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
    ELSE 'Premium'
END AS price_tier
FROM products;

```

Output:

	product_name character varying (100) 	unit_price numeric (10,2) 	price_tier text 
1	Wireless Earbuds	1499.00	Mid-Range
2	Cotton T-Shirt	799.00	Budget
3	Smart Watch	2999.00	Mid-Range
4	Running Shoes	4599.00	Premium
5	Bluetooth Speaker	3499.00	Premium
6	Bedsheet Set	1299.00	Mid-Range
7	Laptop Stand	899.00	Budget
8	Cushion Covers (Set)	599.00	Budget

```

-- Q25
SELECT
SUM(
CASE

```

```

        WHEN status = 'Delivered' THEN 1
        ELSE 0
    END
) AS delivered_orders,

SUM(
    CASE
        WHEN status <> 'Delivered' THEN 1
        ELSE 0
    END
) AS not_delivered_orders
FROM orders;
Output:

```

	delivered_orders bigint 	not_delivered_orders bigint 
1	6	4

```

-- Q26
-- ACID Properties
-- A - Atomicity:All operations succeed or all fail.
--
-- C - Consistency:Database remains valid before and after transaction.
--
-- I - Isolation:Concurrent transactions do not interfere.
--
-- D - Durability:Committed data remains saved even after system failure.
--
-- Bank Transfer Example:
-- Money deducted from Account A and added to Account B.
-- If one step fails, entire transaction rolls back.

```

```

-- Q27

```

```

BEGIN;

```

```

INSERT INTO orders
(
    order_id,
    customer_id,
    order_date,
    status,
    total_amount
)
VALUES
(
    1011,
    102,
    CURRENT_DATE,
    'Pending',
    1598.00
);

```

```

INSERT INTO order_items
VALUES

```

```
(
5016,
1011,
202,
1,
799.00,
0
);

INSERT INTO order_items
VALUES
(
5017,
1011,
208,
1,
599.00,
0
);

UPDATE products
SET stock_qty = stock_qty - 1
WHERE product_id = 202;

UPDATE products
SET stock_qty = stock_qty - 1
WHERE product_id = 208;

COMMIT;

-- If any statement fails:
-- ROLLBACK;
```

Output:

```
COMMIT
```